

Pasul 2 - Toate tipurile de cautare la KR

Teorie, pasi, tabele si exemple rezolvate detaliat

Scopul acestui PDF

Dupa Pasul 1, in care ai invatat sa formulezi problema, acum inveti cum rezolvi efectiv cautarea: ce nod alegi din frontiera, ce pui in OPEN, ce inseamna CLOSED, cand algoritmul este complet, cand este optim si cum scrii pasii in examen.

Contents

1	Imaginea de ansamblu: ce inseamna cautarea	2
1.1	Nod, stare, drum, cost	2
1.2	Frontiera / Open List	2
1.3	Closed List	2
1.4	Schema generala TREE-SEARCH	2
1.5	Schema generala GRAPH-SEARCH	3
2	Criteriile dupa care compari algoritmi	3
3	Tabel rapid cu toate cautarile importante	3
4	Exemplul standard pe graf pentru examen	5
5	BFS - Breadth-First Search / Cautare in latime	5
5.1	Ideea	5
5.2	Structura folosita	5
5.3	Cum scrii BFS la examen	5
5.4	Ce garanteaza BFS	6
5.5	Greseala frecventa	6
6	DFS - Depth-First Search / Cautare in adancime	6
6.1	Ideea	6
6.2	Structura folosita	6
6.3	Cum scrii DFS la examen	6
6.4	Ce garanteaza DFS	6
6.5	Greseala frecventa	7
7	DLS - Depth-Limited Search / Cautare in adancime limitata	7
7.1	Ideea	7
7.2	Exemplu simplu	7
7.3	Ce garanteaza DLS	7
8	IDS - Iterative Deepening Search / Cautare iterativa in adancime	7
8.1	Ideea	7
8.2	Exemplu pas cu pas	8
8.3	De ce nu e prea scump ca repeta noduri?	8
8.4	Ce garanteaza IDS	8
9	UCS - Uniform-Cost Search / Cautare uniforma dupa cost	8
9.1	Ideea	8
9.2	Structura folosita	8
9.3	Exemplu pe graful standard	9
9.4	Ce garanteaza UCS	9
10	Cautare bidirectionala	9
10.1	Ideea	9
10.2	Cand se poate folosi	9
10.3	Exemplu	10
10.4	Intuitie complexitate	10

11 Euristici: baza cautarii informate	10
11.1 Ce este o euristica	10
11.2 Euristica admisibila	10
11.3 Euristica consistenta	10
11.4 Euristici uzuale in grid	11
12 Greedy Best-First Search	11
12.1 Ideea	11
12.2 Exemplu pe graful standard	11
12.3 De ce poate fi gresit	11
12.4 Ce garanteaza Greedy	11
13 A* - A star Search	11
13.1 Ideea	11
13.2 Exemplu pas cu pas pe graful standard	12
13.3 Optimalitate	12
13.4 Cazuri limita	12
14 IDA* - Iterative Deepening A*	13
14.1 Ideea	13
14.2 Pasi	13
14.3 Exemplu mic	13
14.4 Avantaje si dezavantaje	13
15 Cautare locala	13
15.1 Ideea	13
15.2 Cand se foloseste	13
15.3 Probleme	14
16 Exercitii tip examen rezolvate	14
16.1 Exercitiul 1: alegerea algoritmului	14
16.2 Exercitiul 2: graf ponderat	14
16.3 Exercitiul 3: ai euristica buna	14
16.4 Exercitiul 4: verificare admisibilitate	14
16.5 Exercitiul 5: completare tabel pentru graful standard	14
17 Cum scrii la examen o problema de cautare pe graf	15
17.1 Exemplu de forma corecta pentru A*	15
18 Asocieri rapide	15
19 Fisa finala de memorat	15

1 Imaginea de ansamblu: ce inseamna cautarea

O problema de cautare a fost deja formulata la Pasul 1 prin: stare initiala, stari finale, actiuni, functie succesor si cost. Cautarea este partea in care algoritmul exploreaza arborele sau graful starilor ca sa gaseasca o solutie.

Ideea centrala: toti algoritmi de cautare au aceeasi schema generala. Diferenta este **ordinea** in care aleg nodul urmator din frontiera.

1.1 Nod, stare, drum, cost

Un **nod** din arborele de cautare nu este doar o stare. De obicei retine:

$$nod = (stare, parinte, actiune, cost).$$

- **stare:** configuratia lumii;
- **parinte:** nodul din care am venit;
- **actiune:** actiunea aplicata pentru a ajunge aici;
- **cost:** costul drumului de la start pana la acest nod, adica $g(n)$.

1.2 Frontiera / Open List

Frontiera sau **OPEN** este multimea nodurilor generate, dar inca neexpandate.

- La BFS, frontiera este o coada FIFO.
- La DFS, frontiera este o stiva LIFO.
- La UCS, Greedy, A*, frontiera este o coada de prioritati.

1.3 Closed List

CLOSED este multimea starilor deja explorate. Se foloseste la **Graph Search**, ca sa nu intri in cicluri si sa nu expandezi aceeasi stare de mai multe ori.

Varianta	Ce face	Cand conteaza
Tree Search	nu memoreaza stari vizitate; aceeași stare poate apărea de mai multe ori	simplic, dar poate explora redundant sau intra in bucle
Graph Search	foloseste CLOSED; o stare este expandata cel mult o data	mai eficient in grafuri cu cicluri, dar consuma memorie

1.4 Schema generala TREE-SEARCH

```

TREE-SEARCH(problema, strategie):
    Open = {nod_start}
    cat timp Open nu este vida:
        nod = alege_din_Open_conform_strategiei(Open)
        daca nod este scop:
            returneaza drumul de la start la nod
        expandeaza nod
        adauga succesorii in Open
    returneaza esec

```

1.5 Schema generala GRAPH-SEARCH

```

GRAPH-SEARCH(problema, strategie):
    Open = {nod_start}
    Closed = multimea vida
    cat timp Open nu este vida:
        nod = alege_din_Open_conform_strategiei(Open)
        daca nod este scop:
            returneaza drumul de la start la nod
        daca starea(nod) este in Closed:
            continua
        adauga starea(nod) in Closed
        expandeaza nod
        adauga succesorii in Open
    returneaza esec

```

2 Criteriile dupa care compari algoritmi

La examen, daca te intreaba proprietati, raspunzi cu aceste patru criterii.

Criteriu	Intrebare la care raspunde
Completitudine	Daca exista solutie, algoritmul garanteaza ca o gaseste?
Optimalitate	Daca exista mai multe solutii, o gaseste pe cea cu cost minim?
Complexitate timp	Cate noduri genereaza/expandea pana gaseste solutia?
Complexitate spatiu	Cata memorie foloseste pentru frontiera si eventual CLOSED?

Notatii folosite:

- b = factor de ramificare;
- d = adancimea celei mai apropiate solutii;
- m = adancimea maxima a arborelui;
- l = limita de adancime la DLS;
- C^* = costul solutiei optime;
- ε = cost minim pozitiv al unei muchii.

3 Tabel rapid cu toate cautarile importante

Algoritm	Alege nodul dupa	Structura	Optim?	Cand il recunosti
BFS	adancime minima	coada FIFO	da, daca toate costurile sunt egale	“pe niveluri”
DFS	adancime maxima	stiva LIFO	nu	“merge pe o ramura pana jos”
DLS	DFS cu limita l	stiva LIFO	nu	“nu cobori sub nivelul l ”
IDS	DLS repetat cu limite 0,1,2,...	stiva repetata	da, daca toate costurile sunt egale	“BFS ca rezultat, DFS ca memorie”
UCS	cost minim $g(n)$	coada de prioritati	da, costuri pozitive	“Dijkstra pe arbore/graf”

Bidirectional	cauta din start si din scop	doua frontiere	da, in cazuri potrivite	“doua cautari care se intalnesc”
Greedy	euristica minima $h(n)$	coada de prioritati	nu	“pare cel mai aproape de scop”
A*	$g(n) + h(n)$ minim	coada de prioritati	da cu conditii pe h	“cost parcurs + estimare”
IDA*	DFS cu prag pe f	stiva	da cu h buna	“A* cu memorie mica”

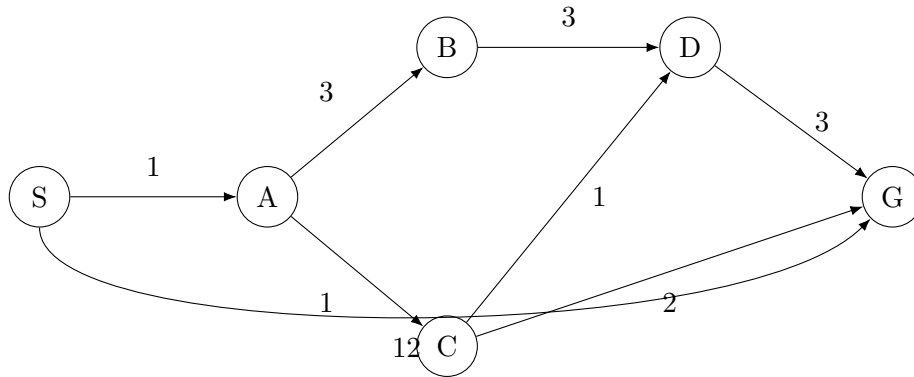
4 Exemplul standard pe graf pentru examen

Vom folosi acelasi graf pentru DFS, BFS, Greedy si A*. Muchiile au costurile:

$$\begin{aligned} S \rightarrow A : 1, \quad S \rightarrow G : 12, \quad A \rightarrow B : 3, \quad A \rightarrow C : 1, \\ B \rightarrow D : 3, \quad C \rightarrow D : 1, \quad C \rightarrow G : 2, \quad D \rightarrow G : 3. \end{aligned}$$

Euristica este:

$$h(S) = 3, \quad h(A) = 2, \quad h(B) = 1, \quad h(C) = 4, \quad h(D) = 2, \quad h(G) = 0.$$



Regula de ordine: cand doi succesori sunt la egalitate sau cand trebuie listati, folosim ordinea lexicografica: A inainte de B, B inainte de C etc. La examen, daca se da alta ordine, respecti enuntul.

5 BFS - Breadth-First Search / Cautare in latime

5.1 Ideea

BFS exploreaza arborele **pe niveluri**. Intai toate nodurile de la adancime 0, apoi toate nodurile de la adancime 1, apoi adancime 2 etc.

5.2 Structura folosita

BFS foloseste o **coada FIFO**. FIFO inseamna: primul intrat este primul scos.

BFS:

```

Open = coada cu nodul start
cat timp Open nu este vida:
    nod = scoate primul nod din coada
    daca nod este scop: returneaza solutia
    genereaza succesorii nodului
    adauga succesorii la finalul cozii
  
```

5.3 Cum scrii BFS la examen

Scrii frontiera dupa fiecare pas. Pentru graful standard:

Rezolvare pas cu pas. Pornim cu:

$$OPEN = [S].$$

Expandam S si generam A si G :

$$OPEN = [S - A, S - G].$$

Daca verificam scopul la generare sau la extragere, $S - G$ este prima solutie de adancime 1. Deci:

$$\boxed{S - G}$$

este solutia BFS.

5.4 Ce garanteaza BFS

- Este complet daca solutia are adancime finita.
- Este optim doar daca toate muchiile au acelasi cost.
- Daca exista costuri diferite, poate gasi un drum cu mai putine muchii, dar mai scump.

5.5 Greseala frecventa

Nu confunda BFS cu UCS. BFS se uita la numarul de muchii, nu la costuri. In exemplul de mai sus, BFS alege $S - G$, desi costul este 12, iar $S - A - C - G$ costa 4.

6 DFS - Depth-First Search / Cautare in adancime

6.1 Ideea

DFS merge cat mai adanc pe prima ramura disponibila. Cand nu mai poate continua, face backtracking.

6.2 Structura folosita

DFS foloseste o **stiva LIFO**. LIFO inseamna: ultimul intrat este primul scos.

DFS:

```
Open = stiva cu nodul start
cat timp Open nu este vida:
    nod = scoate ultimul nod adaugat
    daca nod este scop: returneaza solutia
    genereaza succesorii
    adauga succesorii in stiva
```

6.3 Cum scrii DFS la examen

Pentru graful standard, in ordinea din model:

Rezolvare pas cu pas.

$$OPEN = [S].$$

Expandam S :

$$OPEN = [S - A, S - G].$$

DFS merge pe ramura $S - A$:

$$OPEN = [S - A - B, S - A - C, S - G].$$

DFS continua pe $S - A - B$:

$$OPEN = [S - A - B - D, S - A - C, S - G].$$

DFS continua pe $S - A - B - D$:

$$OPEN = [S - A - B - D - G, S - A - C, S - G].$$

Primul scop gasit este:

$$\boxed{S - A - B - D - G}.$$

6.4 Ce garanteaza DFS

- Nu este complet in spatii infinite sau daca exista cicluri fara CLOSED.
- Nu este optim.
- Are memorie mult mai mica decat BFS: memoreaza drumul curent si fratii neexplorati.

6.5 Greseala frecventa

Daca exista deja un scop aproape de start, DFS tot poate merge pe o ramura lunga inainte sa-l gaseasca. De aceea DFS nu garanteaza drum minim.

7 DLS - Depth-Limited Search / Cautare in adancime limitata

7.1 Ideea

DLS este DFS, dar nu ai voie sa cobori sub o limita l .

```
DLS(nod, limita):
    daca nod este scop: returneaza solutie
    daca limita == 0: returneaza cutoff
    pentru fiecare succesor:
        ruleaza DLS(succesor, limita - 1)
```

7.2 Exemplu simplu

Folosim graful standard. Scopul G exista la adancime 1 prin drumul $S - G$ si la adancime 3 prin $S - A - C - G$.

DLS cu limita $l = 0$

Se verifica doar S . Cum S nu este scop, rezultatul este cutoff/esec.

DLS cu limita $l = 1$

Se verifica S , apoi succesorii A si G :

$$S \Rightarrow S - A, S - G.$$

Gasim scopul:

$$\boxed{S - G}.$$

DLS cu limita prea mica

Daca singura solutie ar fi la adancime 3 si alegem $l = 2$, algoritmul nu o gaseste. Deci DLS nu este complet daca limita nu este suficienta.

7.3 Ce garanteaza DLS

- Este complet doar daca limita este cel putin adancimea unei solutii si arborele este finit pana la limita.
- Nu este optim.
- Memoria este mica, ca la DFS.

8 IDS - Iterative Deepening Search / Cautare iterativa in adancime

8.1 Ideea

IDS ruleaza DLS de mai multe ori, crescand limita:

$$l = 0, 1, 2, 3, \dots$$

Practic, obtine avantajul BFS la gasirea celei mai apropiate solutii, dar foloseste memorie de DFS.

IDS:

```

    pentru limita = 0, 1, 2, ...:
        rezultat = DLS(start, limita)
        daca rezultat este solutie:
            returneaza rezultat

```

8.2 Exemplu pas cu pas

Avem arborele:

$$S \rightarrow A, B, \quad A \rightarrow C, D, \quad B \rightarrow E, G.$$

Scopul este G la adancime 2.

Iteratia 0

Limita 0: verificam doar S . Nu e scop.

Iteratia 1

Limita 1: verificam S , apoi A, B . Nu gasim G .

Iteratia 2

Limita 2: verificam S , apoi A, B , apoi copiii lor. Gasim G .

$$\boxed{S - B - G}.$$

8.3 De ce nu e prea scump ca repeta noduri?

Desi IDS reexpandeaza noduri, majoritatea nodurilor dintr-un arbore sunt pe ultimul nivel. Repetarea nivelurilor mici nu domina costul total.

8.4 Ce garanteaza IDS

- Este complet daca b este finit.
- Este optim daca toate costurile muchiilor sunt egale.
- Are spatiu mic, aproximativ ca DFS.

9 UCS - Uniform-Cost Search / Cautare uniforma dupa cost

9.1 Ideea

UCS alege mereu nodul cu cel mai mic cost acumulat de la start:

$$f(n) = g(n).$$

Este echivalent cu Dijkstra aplicat pe arbore/graf de cautare.

9.2 Structura folosita

UCS foloseste o coada de prioritati ordonata crescator dupa $g(n)$.

UCS:

```

Open = priority queue dupa g(n)
cat timp Open nu este vida:
    nod = scoate nodul cu g minim
    daca nod este scop: returneaza solutia
    expandeaza nod
    adauga succesorii cu costurile lor actualizate

```

9.3 Exemplu pe graful standard

Aici ignoram h , folosim doar costurile.

Rezolvare pas cu pas. Start:

$$OPEN = [S(0)].$$

Expandam S :

$$OPEN = [S - A(1), S - G(12)].$$

Scoatem $S - A(1)$:

$$OPEN = [S - A - C(2), S - A - B(4), S - G(12)].$$

Scoatem $S - A - C(2)$:

$$OPEN = [S - A - C - D(3), S - A - C - G(4), S - A - B(4), S - G(12)].$$

Scoatem $S - A - C - D(3)$:

$$OPEN = [S - A - C - G(4), S - A - B(4), S - A - C - D - G(6), S - G(12)].$$

Urmatorul scop cu cost minim este:

$$\boxed{S - A - C - G}, \quad cost = 4.$$

9.4 Ce garanteaza UCS

- Este complet daca fiecare arc are cost cel putin $\varepsilon > 0$.
- Este optim pentru costuri pozitive.
- Poate consuma multa memorie, pentru ca retine multe drumuri in OPEN.

10 Cautare bidirectionala

10.1 Ideea

In loc sa cauti doar din start spre scop, cauti simultan:

- din start spre scop;
- din scop spre start.

Cautarea se opreste cand cele doua frontiere se intalnesc.

10.2 Cand se poate folosi

Este buna daca:

- starea scop este cunoscuta explicit;
- actiunile se pot inversa;
- poti testa usor daca cele doua frontiere au intersectie.

10.3 Exemplu

Drumul real este:

$$S - A - C - G.$$

Cautare din start:

$$S \Rightarrow A.$$

Cautare din scop:

$$G \Rightarrow C.$$

Urmatorul nivel:

$$A \Rightarrow C.$$

Cele doua cautari se intalnesc in C , deci reconstruim:

$$\boxed{S - A - C - G}.$$

10.4 Intuitie complexitate

Daca solutia este la adancime d , in loc sa cauti aproximativ b^d noduri, cauti doua frontiere de adancime $d/2$, deci aproximativ:

$$O(b^{d/2}).$$

11 Euristici: baza cautarii informate

11.1 Ce este o euristica

O euristica $h(n)$ estimeaza costul minim ramas de la nodul n la scop.

Reguli:

- $h(n) \geq 0$;
- $h(scop) = 0$;
- trebuie sa fie specifica problemei;
- exemple: Manhattan, Euclidian, Chebyshev, Octile.

11.2 Euristica admisibila

O euristica este admisibila daca nu supraestimeaza niciodata costul real:

$$h(n) \leq h^*(n), \quad \forall n.$$

Ca sa arati ca o euristica NU este admisibila, gasesti un singur contraexemplu: un nod n cu $h(n) > h^*(n)$.

11.3 Euristica consistenta

O euristica este consistenta daca pentru orice muchie (n, n') :

$$h(n) \leq c(n, n') + h(n').$$

Intuitiv: euristica nu are voie sa scada prea brusc intre doua noduri vecine.

11.4 Euristici uzuale in grid

Euristica	Formula	Cand se foloseste
Manhattan	$ x - x_g + y - y_g $	miscari in 4 directii, cost unitar
Euclidiana	$\sqrt{(x - x_g)^2 + (y - y_g)^2}$	spatiu continuu sau drum liber
Chebyshev	$\max(dx , dy)$	8 directii, cost unitar
Octile	$\max(dx, dy) + (\sqrt{2} - 1) \min(dx, dy)$	8 directii, diagonala costa $\sqrt{2}$

12 Greedy Best-First Search

12.1 Ideea

Greedy alege nodul care pare cel mai aproape de scop dupa euristica:

$$f(n) = h(n).$$

Nu il intereseaza cat a costat drumul pana aici.

Greedy:

```

Open = priority queue dupa h(n)
cat timp Open nu este vida:
    nod = scoate nodul cu h minim
    daca nod este scop: returneaza solutia
    expandeaza nod
    adauga succesorii in Open

```

12.2 Exemplu pe graful standard

Rezolvare pas cu pas. Start:

$$S(3).$$

Expandam S :

$$S - A(2), \quad S - G(0).$$

Greedy alege G , pentru ca $h(G) = 0$ este minim.

$$\boxed{S - G}.$$

12.3 De ce poate fi gresit

Drumul $S - G$ are cost 12. Dar exista drumul $S - A - C - G$ de cost 4. Greedy a fost atras de scop deoarece se uita doar la h , nu si la g .

12.4 Ce garanteaza Greedy

- Nu este optim.
- Nu este complet in general, mai ales in grafuri infinite sau cu cicluri fara CLOSED.
- Poate fi rapid daca euristica este buna.

13 A* - A star Search

13.1 Ideea

A* combina costul parcurs cu estimarea ramasa:

$$f(n) = g(n) + h(n).$$

- $g(n)$ = cat am platit deja de la start la n ;
- $h(n)$ = cat estimez ca mai am pana la scop;
- $f(n)$ = estimarea costului total daca merg prin n .

A*:

```

Open = priority queue dupa  $f(n) = g(n) + h(n)$ 
cat timp Open nu este vida:
    nod = scoate nodul cu  $f$  minim
    daca nod este scop: returneaza solutia
    expandeaza nod
    pentru fiecare succesor:
        calculeaza  $g, h, f$ 
        adauga in Open

```

13.2 Exemplu pas cu pas pe graful standard

Rezolvare pas cu pas. Start:

$$S : f = 0 + 3 = 3.$$

Expandam S :

$$S - A : f = 1 + 2 = 3, \quad S - G : f = 12 + 0 = 12.$$

Alegem $S - A$, pentru ca are $f = 3$.

Expandam $S - A$:

$$S - A - B : f = 4 + 1 = 5,$$

$$S - A - C : f = 2 + 4 = 6,$$

$$S - G : f = 12.$$

Alegem $S - A - B$, pentru ca are $f = 5$.

Expandam $S - A - B$:

$$S - A - B - D : f = 7 + 2 = 9,$$

$$S - A - C : f = 6, \quad S - G : f = 12.$$

Alegem $S - A - C$, pentru ca are $f = 6$.

Expandam $S - A - C$:

$$S - A - C - D : f = 3 + 2 = 5,$$

$$S - A - C - G : f = 4 + 0 = 4.$$

Acum cel mai mic f este scopul $S - A - C - G$:

$$\boxed{S - A - C - G}, \quad cost = 4.$$

13.3 Optimalitate

Varianta	Cand este optima
A* Tree Search	daca h este admisibila
A* Graph Search	daca h este consistenta

13.4 Cazuri limita

- Daca $h(n) = 0$, atunci A* devine UCS.
- Daca $h(n) = h^*(n)$, atunci euristica este perfecta si A* merge direct pe drumul optim, dar in practica nu putem calcula usor h^* .
- Daca folosim $f(n) = g(n) + w \cdot h(n)$, obtinem Weighted A*. Pentru $w = 0$ seamana cu UCS, pentru $w \rightarrow \infty$ seamana cu Greedy.

14 IDA* - Iterative Deepening A*

14.1 Ideea

IDA* combina memoria mica de la DFS/IDS cu functia de evaluare de la A*.

Nu limiteaza adancimea, ci limiteaza valoarea:

$$f(n) = g(n) + h(n).$$

14.2 Pasi

1. Prag initial:

$$threshold = f(start) = h(start).$$

2. Ruleaza DFS, dar expandeaza doar nodurile cu:

$$f(n) \leq threshold.$$

3. Daca nu gasesti scopul, memorezi cel mai mic $f(n)$ care a depasit pragul.
4. Noul prag devine acea valoare minima depasita.
5. Repeti pana gasesti solutia.

14.3 Exemplu mic

Presupunem:

$$f(S) = 3.$$

Prima iteratie are prag 3. Daca in DFS apar noduri cu $f = 5, 6, 12$, ele nu sunt expandate, dar retinem cel mai mic prag depasit: 5.

A doua iteratie are prag 5. Daca apare un scop cu $f = 4$, il gasim. Daca nu, crestem iar pragul.

14.4 Avantaje si dezavantaje

- Avantaj: memorie mica, ca DFS.
- Avantaj: poate gasi solutie optima cu euristica admisibila/consistenta.
- Dezavantaj: reexpandeaza noduri, deci poate fi mai lent decat A*.

15 Cautare locala

15.1 Ideea

Cautarea locala nu retine drumul de la start la scop. Retine doar o stare curenta si o imbunatateste.

```
Hill-Climbing:
  stare = stare initiala
  repeta:
    vecini = succesorii(stare)
    cel_mai_bun = vecinul cu valoarea cea mai buna
    daca cel_mai_bun nu imbunatateste starea:
      returneaza stare
    stare = cel_mai_bun
```

15.2 Cand se foloseste

- cand spatiul starilor este foarte mare;
- cand nu te intereseaza neaparat drumul, ci doar o stare buna;
- in optimizare, de exemplu N-regine.

15.3 Probleme

- se poate bloca in optim local;
- poate ramane pe platou;
- nu este complet si nu este optim in general.

16 Exerciitii tip examen rezolvate

16.1 Exerciitiul 1: alegerea algoritmului

Ai un graf neponderat si vrei numarul minim de mutari pana la scop. Ce alegi?

Rezolvare pas cu pas. Aleg BFS, deoarece toate costurile sunt egale si BFS gaseste solutia de adancime minima. Daca toate muchiile au cost 1, adancimea minima inseamna cost minim.

16.2 Exerciitiul 2: graf ponderat

Ai costuri diferite pe muchii si vrei solutia de cost minim. Nu ai euristica. Ce alegi?

Rezolvare pas cu pas. Aleg UCS, deoarece UCS extrage mereu nodul cu $g(n)$ minim si este optim pentru costuri pozitive.

16.3 Exerciitiul 3: ai euristica buna

Ai costuri diferite si o euristica admisibila. Ce alegi?

Rezolvare pas cu pas. Aleg A*. Folosesc:

$$f(n) = g(n) + h(n).$$

Daca folosesc Tree Search, admisibilitatea lui h este suficienta pentru optimalitate. Daca folosesc Graph Search, am nevoie de consistenta.

16.4 Exerciitiul 4: verificare admisibilitate

Se da $h(C) = 4$, iar costul real minim de la C la scop este $h^*(C) = 2$. Este h admisibila?

Rezolvare pas cu pas. Nu. Pentru admisibilitate trebuie:

$$h(C) \leq h^*(C).$$

Dar:

$$4 > 2.$$

Deci h supraestimeaza costul real pentru C si nu este admisibila.

16.5 Exerciitiul 5: completare tabel pentru graful standard

Completeaza solutiile gasite de fiecare algoritm pe graful standard.

Algoritm	Solutie gasita	Cost
DFS	$S - A - B - D - G$	$1 + 3 + 3 + 3 = 10$
BFS	$S - G$	12
Greedy	$S - G$	12
UCS	$S - A - C - G$	$1 + 1 + 2 = 4$
A*	$S - A - C - G$	4

17 Cum scrii la examen o problema de cautare pe graf

Sablon de raspuns:

1. Scriu regula algoritmului: ce alege din frontiera.
2. Scriu frontiera initiala: $[S]$.
3. La fiecare pas: expand nodul ales si listez noua frontiera.
4. Daca e Greedy, langa fiecare nod scriu h .
5. Daca e UCS, langa fiecare nod scriu g .
6. Daca e A^* , langa fiecare nod scriu $g + h$.
7. Cand ajung la scop, scriu drumul si costul.

17.1 Exemplu de forma corecta pentru A^*

Nu scrie doar: “ A^* alege drumul bun”. Trebuie sa arati valorile.

Forma buna:

$$\begin{aligned}
 &S(0 + 3 = 3) \\
 &S - A(1 + 2 = 3), \quad S - G(12 + 0 = 12) \\
 &S - A - B(4 + 1 = 5), \quad S - A - C(2 + 4 = 6), \quad S - G(12) \\
 &S - A - B - D(7 + 2 = 9), \quad S - A - C(6), \quad S - G(12) \\
 &S - A - C - D(3 + 2 = 5), \quad S - A - C - G(4), \quad S - A - B - D(9), \quad S - G(12) \\
 &\boxed{S - A - C - G, \text{ cost} = 4.}
 \end{aligned}$$

18 Asocieri rapide

Daca vezi in cerinta...	Te gandesti la...
“pe niveluri”	BFS
“cel mai adanc nod”	DFS
“limita de adancime”	DLS
“limite crescatoare 0,1,2,...”	IDS
“cost minim acumulat”	UCS
“din start si din scop”	Bidirectional Search
“ $h(n)$ minim”	Greedy
“ $g(n)+h(n)$ minim”	A^*
“ h admisibila”	nu supraestimeaza costul real
“ h consistenta”	respecta $h(n) \leq c(n, n') + h(n')$
“memorie mica, dar cu euristica”	IDA*
“spatiu mare, optimizare, stare buna”	cautare locala / hill climbing

19 Fisa finala de memorat

BFS

- FIFO.
- Pe niveluri.
- Complet.
- Optim daca toate costurile sunt egale.

DFS

- LIFO.
- Merge adanc.
- Memorie mica.
- Nu este optim.

DLS

- DFS cu limita.
- Poate rata solutia.

IDS

- DLS cu limite crescatoare.
- Complet.
- Optim pentru costuri egale.

UCS

- Prioritate dupa $g(n)$.
- Optim pentru costuri pozitive.

Greedy

- Prioritate dupa $h(n)$.
- Rapid, dar nu optim.

A*

- Prioritate dupa $g(n) + h(n)$.
- Tree optim cu h admisibila.
- Graph optim cu h consistenta.

IDA*

- DFS cu prag pe f .
- Memorie mica.

Surse folosite

Material sintetizat din cursurile de IA pentru partea de Rezolvarea problemelor si cautare: Curs 2, Curs 3, Curs 4, Curs 5 si modelul de examen/rezolvarea modelului. In special: TREE-SEARCH/GRAPH-SEARCH, Open/Closed List, BFS/DFS/UCS/IDS/Bidirectional, Greedy, euristici, A*, IDA* si exemplul de examen cu graful S, A, B, C, D, G .